

GraphRAG-Enhanced Architectural Compliance via Conflict-Aware Constraint Traversal

Ravicha Suksawasdi Na Ayuthaya (z5625225)

Supervisor: Jianwei Wang

Assessor: Zhengyi Yang

School of Computer Science and Engineering, UNSW

July 2026

Abstract

Architectural Decision Records (ADRs) explain why software teams make certain structural choices, but because these documents recorded in natural languages, automated tools cannot check them. As systems grow and developers leave, the gap between the original plan and the actual code widens. Researchers call this “architectural drift.” AI coding tools make this problem worse. They generate code faster, but the reasoning behind that code is often a guess rather than a deliberate choice.

This report presents a system that checks whether new code changes follow the rules written in plain-language ADRs. No single component is novel: Tree-sitter extracts syntax, LLMs extract constraints from prose, BFS computes reachability, and declarative conflict resolution orderings are well established. The contribution is the composition: an end-to-end pipeline that ingests ADRs and source code into an Architectural Decision Graph (ADG), scopes checks to a PR diff, traverses constraint edges to detect both direct and transitive violations, and resolves conflicts through a deterministic priority ordering (R1: supersession, R2: specificity, R3: recency, R4: human). Each violation report carries the full traversal path, the rule applied, and the rationale. No prior system combines diff-scoped graph traversal, declarative constraint triples with both positive and negative polarity, and deterministic conflict resolution over ADR-linked code.

The literature review looks at past work on ADRs, architectural drift, compliance tools, knowledge graphs, and conflicting rules. It finds a gap that no existing system fills: checking just the new code changes, searching the entire code graph, using rules that say what to do and what not to do, and settling rule conflicts in a fully predictable way. The report proposes a way to evaluate CPT by testing it against a standard AI model on real codebases that use ADRs, measuring how accurate it is, how many violations it catches, and what kinds of violations it finds.

Contents

1	Introduction	3
2	Literature Review	4
2.1	Architectural Decision Records and Drift	4
2.2	Static Analysis and Architectural Compliance	5
2.3	Knowledge Graphs and Code Retrieval	6
2.3.1	Graph Structure	6
2.3.2	Bridging Natural-Language ADRs to the Code Graph	6
2.3.3	Constraint Path Traversal and Graph Reachability	6
2.4	AI-Assisted Development and Compliance	7
2.5	Constraint Conflict Resolution	7
2.6	Dependency Role Classification and Noise Suppression	7
3	Theoretical Framework and Algorithm Design	8
3.1	The Architectural Decision Graph (ADG)	8
3.2	Constraint Path Traversal (CPT)	8
3.3	Conflict Resolution: R1-R4	10
3.4	Dependency Role Classification	10
3.5	The Novelty Claim	10
4	Implementation	11
4.1	System Architecture	11
4.2	Symbolic Constraint Resolution	11
4.3	CPT Detection Engine	12
4.4	Resolution and Dismissal	12
4.5	Specificity Computation	12
5	Evaluation Framework	12
5.1	Research Questions	12
5.2	Evaluation Design	13
5.2.1	Subtask 2: CPT Detection vs Pi+LLM Baseline	13
5.2.2	Subtask 3: Ablation of Optimisations	13
5.2.3	Dataset	13
6	Planned Further Work	14
7	Conclusion	14
	References	15

1 Introduction

Software architecture is about both the structure of a system and the decisions behind it [1], [2]. When those decisions are documented at all, the documentation usually drifts away from what the code actually does, and the two are rarely brought back in line [1]. This gap, called architectural drift or erosion, makes systems harder to maintain and evolve [1], [2].

Architecture Decision Records (ADRs) are a lightweight fix: one short document per decision, kept alongside the code, so the rationale survives when developers leave. Their use on GitHub is growing year over year [3]. But ADRs are prose, invisible to automated checks, so the loop between what was decided and what was committed never closes on its own.

AI-assisted development makes this worse. Code generators and coding agents speed up the rate at which changes land, and the reasoning behind a change is increasingly a guess rather than a deliberate choice. Manual consistency checks that were already unsustainable for human-driven development [2] become infeasible at the pace AI imposes. What was a slow drift becomes a fast one, and the “who decided this” trail that ADRs were meant to preserve is exactly the trail that AI-generated code does not produce.

This work closes that loop. It builds an Architectural Decision Graph (ADG) from source code and ADR documents, then uses Constraint Path Traversal (CPT) to detect violations of architectural constraints at PR review time. Conflicting constraints are resolved through a deterministic priority ordering (R1:R4) that reduces false positives, and every violation report includes the full traversal path, the rule applied, and the rationale.

The original thesis topic framed this work as GraphRAG-Enhanced Memory Management for Intelligent Agents, with three subtasks: graph-based memory representation, relation-aware memory retrieval, and selective forgetting. Table 1 shows how these subtasks map to this work’s design.

Original Subtask	This Research Reframe
Graph-Based Memory Representation	ADG: Unified graph encoding syntactic structure + semantic constraints.
Relation-Aware Memory Retrieval	CPT traversal: walks from code diff, detecting violation via constraint-edge traversal.
Selective Forgetting and Memory Optimization	R1-R4 conflict resolution as false-positive reduction. Incremental graph computation, MDS pruning, crystallization, and version-based pruning are planned future literature review topics , not yet specified.

Table 1: Mapping from the original thesis subtasks to the design.

The research question is whether deterministic graph retrieval over ADRs can provide precise, recallable architectural compliance at PR review time, and what open problems remain for AI-assisted development. This report surveys the relevant literature (Section 2), presents the theoretical framework and algorithm design (Section 3), describes the implementation (Section 4), outlines the evaluation framework (Section 5), and details the plan for COMP9992 (Section 6).

2 Literature Review

The literature relevant to this work spans five thematic areas: ADRs and architectural drift, static analysis and compliance tools, knowledge graphs and code retrieval, constraint conflict resolution, and transitive violation detection. Each area is surveyed below with emphasis on what prior work provides and where the gap lies.

2.1 Architectural Decision Records and Drift

An ADR captures one architectural decision (context, decision, status, consequences) in a short document. Nygard’s template dominates on GitHub; structured variants such as MADR add fields for options and trade-offs [3]. ADRs are lightweight and kept alongside code, but they remain prose: invisible to automated compliance checks.

Drift is the gradual divergence of implementation from documented architecture; erosion is the more severe case where undocumented changes degrade the architecture itself. Empirical studies link drift to delayed maintenance and knowledge loss [1], [2]. The recurring finding is that documentation and code evolve on separate timelines: decisions recorded late or never, rarely reconciled.

Three datasets underpin this work: Buchgeher et al. [3] catalogue 6,362 ADRs across 921 repos; ADRMiner [4] adds text mining across 4,326 ADRs in 547 repos; Su et al. [5] provide 109 repos and 980 ADRs, pre-filtered for quality, representing the strongest work on LLM-based ADR violation detection (strong on explicit violations, weak on implicit ones).

2.2 Static Analysis and Architectural Compliance

Tool	What it detects	Limitation	What this work addresses
SonarQube	Code quality rules, code smells, security hotspots	Symptom-level; not decision-aware; no ADR memory	Decision-graph retrieval, not rule-pattern matching
ArchUnit	Layer, cycle, inheritance, naming constraints as tests	Hand-coded rules; no ADR link; no conflict resolution	ADR-linked graph with deterministic conflict resolution
CodeRabbit	Natural-language diff comments	Probabilistic; no decision memory; high noise	Persistent ADR memory; traversable decision path
ArchLintor [6]	Declarative forbidden/required rules with absence detection	Whole-project static check; no diff scope; direct dependency check only	Diff-scoped per-commit evaluation with reachability over the code graph
ArdoCo [7]	LLM-recovered traceability links between arch docs and code FQNs	Recovers links only; does not enforce ADR constraints at CI	Borrows ArdoCo for seeding, then enforces ADR constraints at commit time
SHACL/SPARQL [8]	Declarative constraints + graph reachability over RDF	RDF triple store only; no code graph; no diff scope; no ADR semantics; no priority-ordered conflict resolution	Same declarative + reachability primitive, applied to a code graph, diff-scoped, ADR-aware, with priority-ordered resolution

Table 2: Comparison of static analysis and compliance tools.

Table 2 surveys six entries across three paradigms: industry linters, academic rule checkers, and declarative graph-constraint standards. SHACL/SPARQL comes closest structurally (declarative + reachability) but lives over RDF, not a diff-scoped code graph, and lacks ADR semantics and priority-ordered conflict resolution. No prior system holds decisions as a retrievable graph checked at CI time with a full traversal path and rationale.

ArchUnit provides declarative DSL rules for layer/onion architectures with both `forbidden` and `requires` polarity, but operates on the full codebase with no diff scoping. Dependency-cruiser is the closest DSL match to ADRLinter’s `PROHIBITS/REQUIRES` triples, supporting `forbidden`, `allowed`, `required`, and `reachable`, but also evaluates over the full graph without diff-driven scoping.

2.3 Knowledge Graphs and Code Retrieval

2.3.1 Graph Structure

The convention that dominates both static-analysis and architecture-compliance literature is the fully qualified name (FQN): a dotted path from root package to module, class, function, or method.

Terra and Valente [1] define a dependency constraint language where every rule references its subjects and objects by qualified name: a typed relation between two FQN patterns with wildcards for hierarchical scope. ADRLinter adopts this shape: a `ConstraintEdge` carries a subject pattern, a predicate, and an object pattern, joined to code through FQN matching. Greifengberg, Mueller, and Rumpe [2] extend the same primitive to plugin-based systems: DepCoL models plugins, packages, and classes as nodes identified by qualified names, with architectural restrictions as typed edges and a consistency checker that walks the graph. Together, [1] and [2] establish the modelling choice this work builds on: *nodes keyed by FQN, code relationships as typed edges, architectural constraints as edges whose endpoints are FQN patterns over those nodes.*

The NL-to-structured-model translation that bridges prose ADRs to this graph rests on Keim et al. [9], who extract typed model elements from natural-language architecture documentation with F1 0.81 on traceability link recovery and 0.93 accuracy on detecting absent model elements.

2.3.2 Bridging Natural-Language ADRs to the Code Graph

ADR constraint endpoints are prose, not FQN patterns. The ArdoCo line addresses this seam: TransArC [10] inserts a component-based Software Architecture Model (SAM) between documentation and code, composing SAD-to-SAM and SAM-to-code trace recovery transitively into SAD-to-code links, reaching weighted-average F1 of 0.87. Fuchss et al. [7] remove the SAM requirement: an LLM extracts component names from the SAD and scores their resemblance to code FQNs through a composable tree of heuristics (name, package, path resemblance, subpackage filters, provided-interface correspondence), reaching F1 0.86 with GPT-4o.

For ADRLinter, the implication is direct: the current `fqn_matches_pattern` binary match should be replaced by a scored heuristic tree of the ArdoCo shape, with standalone heuristics returning confidence per (ADR endpoint pattern, code FQN) tuple, dependent heuristics filtering on graph structure, and aggregations composing them.

2.3.3 Constraint Path Traversal and Graph Reachability

The current ADRLinter implementation computes reachability over the full code-graph adjacency for every commit: BFS from each subject FQN to determine whether a prohibited or required dependency path exists. This is correct but unoptimised. The literature on incremental and bounded graph computation, surveyed below, is a planned future review to determine whether the full-graph cost can be reduced without sacrificing precision.

Fan, Hu, and Tian [11] establish the boundedness framework for incremental graph computation: when an incremental algorithm’s cost is polynomial in the size of the change alone, the computation is *bounded*. Fan et al. [12] give complexity results for graph simulation (optimal linear-time for unit updates), bounded simulation (unbounded in general), and subgraph isomorphism (intractable and unbounded). The 2-hop labelling approach on SCC-condensed DAGs reduces reachability queries to set intersection, providing a natural scalability path if BFS proves to be the bottleneck in practice.

Seedat et al. [13] propose explicit bounded k-hop horizons with per-hop decay for change impact analysis, the closest prior art to diff-scoped graph traversal. The gap is that Seedat propagates an impact set, not a downstream architectural rule check.

An earlier CPT design used a bounded k-hop BFS bubble around each changed FQN. This was identified as unsound: a PROHIBITS constraint whose object sits 4+ hops away is filtered out before any reachability check, producing false negatives, and a REQUIRES constraint whose only path passes through a node outside the bubble produces false positives. The current implementation removes the bubble and uses unbounded BFS on the full adjacency, with multi-source BFS collapsing the pairwise factor for PROHIBITS constraints.

2.4 AI-Assisted Development and Compliance

The Sentinel system [14] is the closest operational comparison to ADRLinter: an autonomous agent that monitors ADR conformance at commit level, achieving 83% true-positive rate. However, Sentinel uses LLM agents for pattern detection without graph traversal, and does not provide deterministic traversal paths or conflict resolution.

Dhaouadi et al. [15] measured contradictory decision pairs in commit messages, finding a 0.29% rate of naturally occurring conflicts. This validates that ADR-ADR conflicts are rare but non-zero, justifying conflict resolution (R1-R4) while confirming that evaluation must supplement natural conflicts with synthetic cases.

The broader landscape includes CodeRabbit (probabilistic diff comments with no decision memory) and LLM-based code review tools that lack persistent ADR-aware memory. The common absence across all approaches is a traversable graph of decisions checked deterministically at CI time.

2.5 Constraint Conflict Resolution

Constraint conflict resolution has mature formalisms in AI planning (constraining plans to remove harmful interactions [16]), constraint programming (nogood recording via “reasoning from last conflicts” [17]), and causal discovery (ASP-based resolution of conflicting statistical constraints [18]).

For ADRLinter, the specific form is: when multiple ADRs impose conflicting constraints on the same code region, which wins? The R1-R4 system (superseded status, specificity, recency, human decision) draws from the specificity ordering in dependency constraint languages [1] and the temporal ordering in ADR status conventions. Mohammadi et al. [19] provide the metric framework: precision, relative coverage, and violation-type breakdown map to the capability and reliability dimensions of their two-dimensional LLM agent evaluation taxonomy.

2.6 Dependency Role Classification and Noise Suppression

Pure graph topology (CALLS, IMPORTS, CONTAINS, INHERITS edges) cannot distinguish `pytest` from `flask`. The literature converges on a two-layer approach: topological graph as a base, semantic role overlay to classify *why* a dependency exists.

Latendresse et al. [20] show that well-known dev tools (`pytest`, `eslint`, `babel-cli`) are never used in production across all studied projects (85.6% of packages are never used in production).

Weeraddana et al. [21] extend this: 92.63% of CI build time wasted on unused dependencies comes from development dependencies. For ADRLinter, `DependencyRole` classification (`DEV_TOOL`, `INFRASTRUCTURE`, `APPLICATION`, `INTERNAL`) at graph construction time, using package metadata, is a lookup problem rather than an inference problem. Filtering `DEV_TOOL` nodes from architectural violation checks directly suppresses the most common class of false positives.

3 Theoretical Framework and Algorithm Design

This section presents the theoretical contributions: the Architectural Decision Graph model, the CPT algorithm, the R1-R4 conflict resolution framework, and the specificity scoring scheme.

3.1 The Architectural Decision Graph (ADG)

Definition 1 (*Architectural Decision Graph*). An ADG is a directed labelled graph $G = (V, E, C)$ where V is a set of FQN nodes (modules, classes, functions, methods, external packages), E is a set of typed structural edges (`CALLS`, `IMPORTS`, `CONTAINS`, `INHERITS`), and C is a set of constraint edges, each a triple (s, p, o) where s is a subject FQN pattern, p is a predicate (`PROHIBITS_DEPENDENCY`, `REQUIRES_DEPENDENCY`, `PROHIBITS_IMPLEMENTATION`, `REQUIRES_IMPLEMENTATION`), and o is an object FQN pattern.

Each constraint edge carries metadata: `specificity` (a numeric score), `adr_id` (provenance), `adr_path` (source file), and `justification` (natural-language rationale). Each FQN node carries a `kind` (module, class, function, method, external) and a `role` (internal, dev_tool, infrastructure, application, unknown).

The dual-track ingestion pipeline constructs the ADG:

Track	Tool	Output
Syntactic	Tree-sitter	FQN nodes, <code>CALLS</code> , <code>IMPORTS</code> , <code>CONTAINS</code> , <code>INHERITS</code> edges
Semantic	LLM + symbolic resolver	ConstraintEdges with source grounding

Table 3: Dual-track ingestion pipeline.

3.2 Constraint Path Traversal (CPT)

The CPT algorithm takes a commit diff, the ADG, and returns a set of violations. The algorithm operates in three phases: constraint matching, structural predicate checking (`PROHIBITS`), and change-triggered predicate checking (`REQUIRES`).


```

def CPT_Detect(D, G):
    changed = ExtractModifiedFQNs(D)
    adj = BuildAdjacency(G.edges)
    matched = MatchConstraints(G)
    violations = set()

    # Phase 1: PROHIBITS (structural)
    for mc in matched:
        if mc.predicate.startswith("prohibits"):
            for s in mc.subject_matches:
                reachable = BFS(s, adj, structural_kinds)
                for o in mc.object_matches:
                    if role(o) != DEV_TOOL:
                        if o in reachable or descendant(o, reachable):
                            violations.add(Violation(mc, s, o))

    # Phase 2: REQUIRES (change-triggered)
    for c in changed:
        for mc in matched:
            if mc.predicate.startswith("requires"):
                if c matches mc.subject or is_descendant:
                    reachable = BFS(c.fqn, adj, structural_kinds)
                    found = any(
                        o in reachable or descendant(o, reachable)
                        for o in mc.object_matches
                        if role(o) != DEV_TOOL
                    )
                if not found:
                    violations.add(Violation(mc, c, o))

    # Phase 3: Resolve conflicts
    violations = R1_R4_Resolve(violations)
    return violations

```

Listing 1: The CPT detection algorithm.

The key insight is that PROHIBITS constraints are *structural*: they hold regardless of what changed in this commit, because they ask “does a forbidden dependency path exist?” REQUIRES constraints are *change-triggered*: they only fire when a changed FQN falls under the constraint’s subject scope, because they ask “does this changed module have a required dependency?”

Definition 2 (*Multi-source reachability*). For a PROHIBITS constraint with subject set S and object set O , multi-source BFS from all nodes in S simultaneously terminates when any node in O is reached. This collapses the pairwise factor from $|S| \times |O|$ BFS calls to one.

The current implementation uses unbounded BFS on the full code-graph adjacency, recomputing reachability for every commit. Incremental and bounded graph computation (2-hop labelling on SCC-condensed DAGs [11]) is planned future work, to be evaluated after measuring wall-clock BFS cost on real repositories.

3.3 Conflict Resolution: R1-R4

When multiple ADRs impose conflicting constraints on the same code region, ADRLinter resolves them through a priority-ordered system:

Rule	Condition	Priority
R1	ADR explicitly supersedes another	Highest
R2	Higher specificity score wins	High
R3	Newer ADR (higher <code>adr_id</code>) wins	Medium
R4	Flag for human review	Default

Table 4: R1-R4 conflict resolution rules, ordered by priority.

Definition 3 (*Specificity score*). The specificity of an FQN pattern is:

$$\text{specificity}(p) = \begin{cases} d(p) + 1 & \text{if } p \text{ is exact (no wildcard)} \\ d(p) & \text{if } p \text{ ends with } .* \end{cases}$$

where $d(p)$ is the dot-depth of the pattern after stripping the `.*` suffix. Examples: `app.db.query` yields 3.0 (exact), `app.services.*` yields 1.5 (one wildcard), `app.*` yields 0.5 (broad wildcard).

The resolution pipeline applies R1-R4 as default system behaviour for false-positive reduction. Ablation of R1-R4 would test whether the system works correctly, not whether it works better. MDS pruning, crystallization, and version-based pruning are planned future literature review topics for memory optimization, not yet specified.

3.4 Dependency Role Classification

The `DependencyRole` attribute on FQN nodes classifies dependencies by their architectural function:

Role	Description	Architecturally relevant
DEV_TOOL	Testing, linting, formatting (pytest, black, mypy)	No
INFRASTRUCTURE	Databases, caches, message queues (redis, elasticsearch)	Context-dependent
APPLICATION	Frameworks and libraries (flask, django)	Yes
INTERNAL	Project modules	Yes

Table 5: Dependency role classification for false-positive suppression.

`DEV_TOOL` nodes are excluded from `PROHIBITS` object matching and `REQUIRES` reachability, directly suppressing the most common class of false positives identified in the dependency literature [20], [21]. `INFRASTRUCTURE` nodes are context-dependent: an ADR that says “the data layer must not depend on the web layer” makes `flask` in the data layer a real violation, not a false positive.

3.5 The Novelty Claim

No prior system combines all four of ADRLinter’s pillars:

System	Diff-driven	Graph tra- versal	Declarative triples	Both polari- ties	Per-commit
ADRLinter	yes	yes	yes	yes	yes
Archy	yes	yes	no	no	yes
dependency- cruiser	no	no	yes	yes	no
ArchUnit	no	no	partial	yes	no
Axiom Refract	no	yes	no	no	no
Seedat et al.	yes	yes	no	no	n/a
Dart Sentinel	yes	no	partial	no	yes

Table 6: Novelty comparison: no prior system combines all four pillars.

Each component has prior art; the combination does not.

4 Implementation

ADRLinter is implemented in Python 3.12, with a FastAPI service layer, Neo4j as the persistent graph store, and Tree-sitter for syntactic parsing. The system is containerised via Docker Compose for reproducibility.

4.1 System Architecture

The pipeline orchestrates the following stages:

Stage	Input	Output
1. Seed	Repository + ADR directory	ADG with structural edges
2. Extract	ADR markdown files	SymbolicConstraints (LLM-assisted)
3. Resolve	SymbolicConstraints + ADG	ConstraintEdges with FQN patterns
4. Merge	ADG + ConstraintEdges	ADG with constraint edges + specificity
5. Diff	Git commit SHA	DiffResult (changed FQNs)
6. Augment	ADG + CommitDiff	Augmented ADG (new edges from diff)
7. Detect (CPT)	DiffResult + ADG	CPTRResult (violations, orphans)
8. Dismiss	CPTRResult + Dismissal list	Filtered violations

Table 7: ADRLinter pipeline stages.

The core data model is defined in `app/services/models.py`. Key types include `FQNNode` (an FQN with kind, role, and source location), `Edge` (typed structural edge), `ConstraintEdge` (subject-predicate-object triple with specificity, ADR provenance, and justification), and `Violation` (a detected violation with matched FQNs, match status, and evidence).

4.2 Symbolic Constraint Resolution

The symbolic resolver (`app/services/adg/symbolic_resolver.py`) bridges the gap between natural-language ADR text and FQN patterns in the code graph. It operates in three stages:

1. *General match*: Exact or wildcard match of `role_general` against module FQNs.
2. *Specific narrow*: Substring-match `role_specific` against descendants of general matches, with priority ordering: exact > prefix overlap > substring containment.
3. *Fallback*: Substring-match `role_specific` against all ADG nodes when general matching fails.

External dependencies (packages not in the code graph) are classified by role using package metadata and known registries, creating EXTERNAL nodes with appropriate `DependencyRole` values.

4.3 CPT Detection Engine

The detection engine (`app/services/cpt/engine.py`) implements the algorithm from Figure Listing 1. Two predicate classes are checked:

- *PROHIBITS (structural)*: For each constraint with a `prohibits_*` predicate, check whether any subject FQN can reach any object FQN via the structural edge kinds. If so, a violation is reported.
- *REQUIRES (change-triggered)*: For each changed FQN and each constraint with a `requires_*` predicate whose subject scope contains the changed FQN, check whether the changed FQN can reach any object FQN. If not, a violation is reported.

Both checks use BFS on the full code-graph adjacency, with `DEV_TOOL` nodes excluded from traversal and matching.

4.4 Resolution and Dismissal

After detection, violations pass through two resolution stages:

1. *Conflict resolution* (`app/services/cpt/resolution.py`): Deduplicate violations, suppress module-level children when the parent already matches, and apply the `suppress_outweighed_prohibits/suppress_outweighed_requires` rules based on specificity scores and ADR recency.
2. *Dismissal* (`app/services/cpt/dismissal.py`): A human-review mechanism where false positives are dismissed and excluded from future reports.

4.5 Specificity Computation

The specificity score is computed in `app/services/pipeline.py`:

$$\text{specificity}(p) = d(p) + \begin{cases} 0 & \text{if } p \text{ ends with } .* \\ 1 & \text{otherwise} \end{cases}$$

This ensures that more specific patterns (e.g., `app.db.query` at 3.0) outweigh broader wildcards (e.g., `app.*` at 0.5) in conflict resolution.

5 Evaluation Framework

5.1 Research Questions

The evaluation addresses three research questions:

1. *RQ1 (Effectiveness)*: Does CPT detect architectural violations that an LLM baseline misses, particularly transitive violations?
2. *RQ2 (Precision)*: What is the precision of CPT-detected violations, measured through human review?
3. *RQ3 (Optimization trade-offs)*: Do optimisations (MDS pruning, crystallization, version-based pruning) trade precision for speed/token savings, and by how much?

5.2 Evaluation Design

5.2.1 Subtask 2: CPT Detection vs Pi+LLM Baseline

The baseline is a Pi agent harness with state-of-the-art LLMs (2-3 models: one frontier, two open/affordable). Pi provides tool access (grep, file reading) with a 20 tool-call cap. Both CPT and Pi receive the same ADR text and repo file tree. The prompt gives no hints about graph traversal or CPT.

Metrics:

Metric	Definition	Notes
Precision	confirmed / total reported	From human review
Relative coverage	violations found by CPT but not LLM, and vice versa	No absolute recall (no oracle). Avoids the oracle problem [19]
Violation type breakdown	direct vs transitive per method	Headline result: CPT should dominate on transitive violations

Table 8: Evaluation metrics for Subtask 2.

Ground truth is established via human review: confirmed violations are true positives, dismissed violations are false positives. This follows the single-annotator pattern common in SE benchmarks (cf. SWE-bench’s gold patch validation) and avoids the annotation burden identified by de Silva and Balasubramaniam [22] as a barrier to architecture conformance checking adoption.

5.2.2 Subtask 3: Ablation of Optimisations

The baseline is the full pipeline with R1-R4 resolution. Ablation tests whether the system works correctly without these optimization. Future optimisations (MDS pruning, crystallization, version-based pruning) are planned literature review topics that will be added as ablation rows once specified:

Arm	Input	Output	Metric
Baseline	Commit diff + repo + ADRs	Code-review agent (Pi) report	Precision, recall, token usage, time taken
CPT, no optimization	diff + repo + ADRs + violation findings	Code-review agent (Pi) report	Precision, recall, token usage, time taken
CPT, with optimization	diff + repo + ADRs + violation findings	Code-review agent (Pi) report	Precision, recall, token usage, time taken

Table 9: Evaluation arms for Subtask 3.

5.2.3 Dataset

A repository qualifies if it has 3+ ADRs with identifiable PROHIBITS/REQUIRES constraints, is a Python codebase, has code-inferable constraints, contains at least one violated ADR, and is publicly available. The target is 5 repositories with 25-50 violation instances.

The primary source is the ADR-Study-Dataset [3] (921 repos, 6,362 ADRs), supplemented by Su et al. [5] (109 repos, 980 ADRs). `python-tuf` (10 ADRs, Nygard format, checkable constraints,

1,712 GitHub stars) is confirmed as the first repository; additional repositories will be selected from the ADR-Study-Dataset filtered for Python and 3+ code-inferable ADRs.

For R1-R4 validation, natural ADR-ADR conflicts are rare (0.29% of pairs [15]). The strategy is to first curate naturally occurring conflicts from real repos, then inject synthetic conflict ADRs if fewer than approximately 10 natural conflicts are found across all repos. Only the conflicting ADR text is synthetic; the code and structural context remain real.

6 Planned Further Work

The remaining work for COMP9992 spans 10 weeks.

1. *Weeks 1-4*: Select 4 additional repositories from the ADR-Study-Dataset (python-tuf confirmed as the first). Design the Pi+LLM prompt template. Pilot ADRLinter on python-tuf. Run the full evaluation: CPT vs Pi+LLM across all 5 repositories. Conduct human review for ground truth annotation.
2. *Weeks 5-7*: Literature review of incremental graph computation, MDS pruning, crystallization, and version-based pruning as potential optimisations. Run ablation of R1-R4 (correctness test). Specify and implement optimisation ablation rows if supported by the literature.
3. *Weeks 8-10*: Replace `fqn_matches_pattern` binary matching with a scored heuristic tree of the ArdoCo shape. Validate R1-R4 conflict resolution on natural and synthetic conflict instances. Write the thesis.

7 Conclusion

This report has presented ADRLinter, a system that closes the loop between architectural decisions recorded as ADRs and the code that implements them. The literature review identified a gap that no prior system fills: the combination of per-commit diff-driven scoping, full-graph traversal with multi-source BFS, declarative constraint triples with both positive and negative polarity, and deterministic conflict resolution. The theoretical framework formalises this gap as the ADG model and the CPT algorithm, with R1-R4 resolution providing false-positive reduction and specificity scoring providing a principled ordering for ambiguous constraints.

The implementation demonstrates that the approach is feasible: the dual-track pipeline (Tree-sitter for syntax, LLM for semantics) constructs the ADG; the symbolic resolver bridges natural-language ADR text to FQN patterns; the CPT engine detects both structural and change-triggered violations; and the resolution pipeline applies R1-R4 with dismissal for human oversight.

The evaluation framework, designed but not yet executed, proposes a direct comparison against an LLM baseline (Pi+LLM) on real-world repositories with ADRs, measuring precision, relative coverage, and violation-type breakdown. The ablation study will first test R1-R4 correctness, then evaluate potential optimisations (MDS pruning, crystallization, version-based pruning) once specified through further literature review.

The remaining 10 weeks of work are clearly defined: benchmark execution, optimisation literature review and ablation, and the heuristic resolution upgrade that replaces binary pattern matching with the scored ArdoCo-style tree.

References

- [1] R. Terra and M. T. Valente, “A dependency constraint language to support object-oriented architectural evolution,” *Software Practice and Experience*, vol. 39, no. 12, pp. 1073–1100, 2009.
- [2] T. Greifenberg, K. Muller, and B. Rumpe, “A domain-specific language for architecture conformance checking,” *arXiv preprint*, 2015.
- [3] G. Buchgeher, R. Riedl, A. Weiss, and S. Zugal, “Using Architecture Decision Records in Open Source Projects: An MSR study on GitHub,” *Proceedings of the International Conference on Software Engineering (MSR track)*, 2023.
- [4] ADRMiner Team, “ADRMiner: Text Mining and Classification of Architecture Decision Records,” *Proceedings of the European Conference on Software Architecture (ECSA)*, 2026.
- [5] L. Su and others, “LLM-based ADR Violation Detection,” *Proceedings of the International Conference on Software Architecture (ICSA)*, 2026.
- [6] D. Resende and others, “ArchLintor: Declarative Architectural Compliance Checking with Absence Detection,” *Brazilian Symposium on Software Engineering (SBES)*, 2024.
- [7] J. Fuchss, J. P. Keim, A. Koziolk, and H. Koziolk, “Enabling Architecture Traceability by LLM-based Architecture Component Name Extraction,” *IEEE Transactions on Software Engineering*, 2025.
- [8] W3C, “SHACL Shapes Constraint Language.” [Online]. Available: <https://www.w3.org/TR/shacl/>
- [9] J. P. Keim, J. Fuchss, A. Koziolk, and H. Koziolk, “Traceability Link Recovery for Software Architecture Documentation,” *Proceedings of the International Conference on Software Architecture (ICSA)*, 2023.
- [10] J. P. Keim, J. Fuchss, A. Koziolk, and H. Koziolk, “Recovering Trace Links Between Software Documentation and Code,” *Proceedings of the International Conference on Software Engineering (ICSE)*, 2024.
- [11] W. Fan, C. Hu, and X. Tian, “Incremental Graph Computations: Doable and Undoable,” *ACM Transactions on Database Systems (TODS)*, 2022.
- [12] W. Fan, J. Li, S. Luo, Z. Tan, X. Wang, and Y. Wu, “Incremental Graph Pattern Matching,” *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2011.
- [13] N. Seedat and others, “Toward Semantically-Seeded, Graph-Propagated Impact Analysis,” *arXiv preprint*, 2024.
- [14] Sentinel Team, “Sentinel: Autonomous Architectural Governance through Asynchronous Agents,” *Proceedings of the International Conference on Software Engineering (ICSE)*, 2025.
- [15] M. Dhaouadi and others, “Contradictory Decision Pairs in Software Development,” *Proceedings of the ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, 2025.

- [16] G. L. Steele and M. Ainsworth, “A Theory of Conflict Resolution in Planning,” *Artificial Intelligence*, 1992.
- [17] C. Lecoutre, “Reasoning from Last Conflict(s) in Constraint Programming,” *Artificial Intelligence*, 2009.
- [18] A. Hyttinen and others, “Constraint-based Causal Discovery: Conflict Resolution with Answer Set Programming,” *Proceedings of the AAAI Conference on Artificial Intelligence*, 2014.
- [19] M. Mohammadi and others, “A Two-Dimensional Taxonomy for LLM Agent Evaluation,” *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2025.
- [20] A. Latendresse and others, “Not All Dependencies are Equal: An Empirical Study on Production Dependencies in NPM,” *Proceedings of the IEEE International Conference on Automated Software Engineering (ASE)*, 2022.
- [21] P. Weeraddana and others, “Dependency-Induced Waste in Continuous Integration: An Empirical Study of Unused Dependencies in the npm Ecosystem,” *Proceedings of the ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, 2024.
- [22] L. de Silva and D. Balasubramaniam, “Architecture Consistency: State of the Practice and Practice of the State,” *IEEE Software*, 2018.